

Детализированный План Реализации и Технический Стек

▢ Ответы и рекомендации

1. Gemini API - отличный выбор для MVP ▢

Согласен на 100%! Вот почему:

Параметр	Gemini Free Tier	GPT-4o	Claude 3.5
Бесплатные запросы	1500 req/день	-	-
RPM (requests/min)	15	Платно	Платно
Контекст	2M tokens	128k	200k
Стоимость (после лимита)	\$0.00 / \$0.075	\$2.50 / \$10	\$3 / \$15

Для тестирования с 10-50 запросами/день:

- ▢ Полностью бесплатно
- ▢ Огромный контекст (можно скармливать большие объемы данных)
- ▢ Gemini 2.0 Flash - отличное качество анализа
- ▢ Нативная интеграция с Google AI Studio для отладки промптов

Рекомендация: ```

MVP: Gemini 2.0 Flash (бесплатно)

→ При масштабировании: Gemini 1.5 Pro (сложный анализ) + Flash (простые задачи)

2. Browser Use vs Browser MCP - уточнение ▢

Признаю неточность! Давай разберем:

Browser MCP (Model Context Protocol)

- ▢ Стандартизированный протокол
 - ▢ Переиспользуемые компоненты
 - ▢ Легкая интеграция между инструментами
 - ▢ Активное развитие (Anthropic)
 - ▢ Относительно новый (меньше примеров)
- ##### **Browser Use** (библиотека)

- ▢ Специализация на AI-управлении браузером
 - ▢ Готовые высокоуровневые абстракции
 - ▢ Хорошая документация
 - ▢ Менее стандартизированный подход
- ##### **Playwright** (классика)

- ▢ Зрелый, стабильный
- ▢ Огромное комьюнити
- ▢ Полный контроль
- ▢ Больше низкоуровневого кода

Моя скорректированная рекомендация для проекта:

Hybrid подход:

1. Основа: Playwright (стабильность, контроль)
2. Интеграция через MCP для стандартизации
3. Обертка AI-логики с использованием паттернов Browser Use

Почему:

- Playwright - production-ready
- MCP - будущее интеграций AI
- Browser Use паттерны - ускорение разработки AI-логики

****Конкретно для парсинга поставщиков:****

```python

# Архитектура

MCP Server (Browser)

↓

Playwright Engine

↓

AI Agent (Gemini) → анализирует DOM → извлекает данные

### 3. VS Code + AI инструменты - идеально! ☐

**Полностью поддерживаю!** Это даст:

#### Отладка промптов и системных инструкций

Workflow:

1. Пишем системный промпт в Continue/Cline
2. Тестируем на реальных задачах парсинга
3. Рефакторим через Gemini Code
4. Сохраняем финальную версию в репозиторий

#### MCP в процессе разработки

VS Code Extensions:

- └─ Continue (MCP клиент)
- └─ Cline (агентный режим)
- └─ MCP Inspector (отладка серверов)

Тестируем:

- MCP сервер для браузера
- MCP сервер для БД (Supabase)
- MCP сервер для файловой системы

#### Рекомендация структуры:

```
.vscode/
├── settings.json # конфиг MCP серверов
├── prompts/
│ ├── parser.md # промпты для парсера
│ ├── analyzer.md # промпты для анализа
│ └── reporter.md # промпты для отчетов
└── mcp-config.json # конфигурация MCP
```

### 4. Только Gemini - правильное решение для MVP ☐

**Аргументы "за":**

#### Упрощение архитектуры

Было (multi-model):

- └─ Claude (анализ)
- └─ GPT-4o (отчеты)
- └─ Gemini (парсинг)

☐ Сложность управления

☐ Разные API

☐ Разные форматы промптов

Стало (Gemini-only):

- └─ Gemini 2.0 Flash/Pro

☐ Один SDK

☐ Единый стиль промптов

☐ Проще отладка

## Специализация Gemini под задачу

# Gemini 2.0 Flash отлично справляется с:

1. Multimodal анализ (скриншоты сайтов)
2. Большие контексты (все данные поставщиков)
3. Structured output (JSON/Pydantic models)
4. Function calling (интеграция с API)

## Пример архитектуры агента

```
from google import genai
```

```
Единый клиент для всех задач
client = genai.Client(api_key=GEMINI_API_KEY)
```

```
Агент 1: Парсинг
parser_agent = client.agentic.create_agent(
 model="gemini-2.0-flash",
 system_instruction="Ты парсишь сайты поставщиков..."
)
```

```
Агент 2: Анализ
analyzer_agent = client.agentic.create_agent(
 model="gemini-2.0-flash",
 system_instruction="Ты анализируешь цены и наличие..."
)
```

```
Агент 3: Отчеты
reporter_agent = client.agentic.create_agent(
 model="gemini-2.0-flash",
 system_instruction="Ты создаешь итоговые рекомендации..."
)
```

## Когда можно добавить другие модели (будущее):

- Специфические задачи, где Gemini слабее
- A/B тестирование качества ответов
- Резервирование при недоступности Gemini

## 5. Инфраструктура - отличная база! 📦

Твоя связка идеальна для быстрого старта:

### Архитектура развертывания

```
docker-compose.yml
```

```
services:
 n8n:
 image: n8nio/n8n
 environment:
 - DB_TYPE=postgresdb
 - DB_POSTGRESDB_HOST=db.supabase.co
 - N8N_ENCRYPTION_KEY=${N8N_KEY}
 volumes:
 - n8n_data:/home/node/.n8n
 labels:
 - "cloudflare-tunnel=n8n.yourdomain.com"
```

```
fastapi:
 build: ./services/api
 environment:
 - GEMINI_API_KEY=${GEMINI_KEY}
 - SUPABASE_URL=${SUPABASE_URL}
 - SUPABASE_KEY=${SUPABASE_KEY}
 labels:
 - "cloudflare-tunnel=api.yourdomain.com"
```

```
playwright:
 build: ./services/scrapper
 volumes:
 - /dev/shm:/dev/shm # для стабильности браузера
 environment:
```

- DISPLAY=:99

```
crawl4ai:
 image: unclecode/crawl4ai
 labels:
 - "cloudflare-tunnel=crawler.yourdomain.com"
```

```
volumes:
 n8n_data:
```

## Cloudflare Tunnel - преимущества

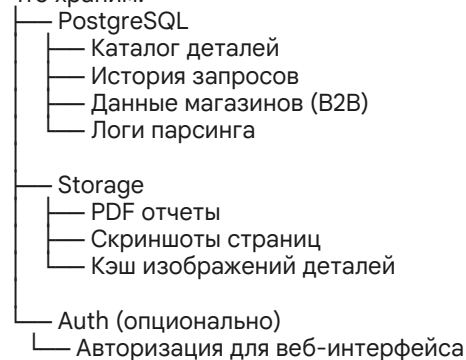
- Нет проблем с динамическим IP
- Автоматический HTTPS
- Защита от DDoS
- Легкая миграция между серверами
- Доступ к контейнерам извне для webhooks

Пример маршрутизации:

```
n8n.partscout.dev → container:5678
api.partscout.dev → container:8000
crawler.partscout.dev → container:8080
```

## Supabase как центральное хранилище

Что храним:



## MCP интеграция

MCP Servers в контейнерах:

1. Supabase MCP Server
  - Доступ к БД через MCP
  - AI может напрямую делать запросы
2. Filesystem MCP Server
  - Работа с PDF/скриншотами
3. Browser MCP Server
  - Playwright через MCP

n8n вызывает MCP → AI обрабатывает → результат в Supabase

## Путь к Kubernetes (будущее)

Когда мигрировать:

- >10 автосервисов (multi-tenancy)
- Нужна автоматическая масштабирование
- Географическая распределенность

Плавная миграция:

Docker Compose → Docker Swarm → K3s → Kubernetes

Твоя текущая архитектура готова к этому!

## 6. B2B - упрощенная схема □

Согласен отложить монетизацию! Фокус на функционале:

## Архитектура Telegram бота (V1)



## Простая реализация (aiogram)

```
bot/b2b_handler.py
```

```
from aiogram import Bot, Dispatcher, types
from aiogram.filters import Command
```

```
Хранилище магазинов (потом в Supabase)
```

```
SHOPS = {
 "shop_a": {"chat_id": 123456, "name": "Автозапчасти А", "distance": 60},
 "shop_b": {"chat_id": 789012, "name": "Детали Б", "distance": 45},
 # ...
}
```

```
MANAGER_CHAT_ID = 999999
```

```
@dp.message(Command("find"))
```

```
async def broadcast_request(message: types.Message, bot: Bot):
```

```
 """Менеджер делает запрос"""
```

```
 query = message.text.replace("/find", "").strip()
```

```
 # Создаем уникальный ID запроса
```

```
 request_id = generate_request_id()
```

```
 # Broadcast всем магазинам
```

```
 for shop_id, shop_data in SHOPS.items():
```

```
 await bot.send_message(
```

```
 chat_id=shop_data["chat_id"],
```

```
 text=f"□ Запрос #{request_id}\n\n{query}\n\n"
```

```
 f"Ответьте: /reply_{request_id} <цена> <наличие>",
```

```
 reply_markup=quick_reply_keyboard(request_id)
```

```
)
```

```

Уведомляем менеджера
await message.answer(
 f"Запрос разослан {len(SHOPS)} магазинам\n"
 f"Ожидайте ответы (макс. 5 минут)"
)

Запускаем таймер агрегации
asyncio.create_task(aggregate_responses(request_id, 300)) # 5 мин

@dp.message(Command("reply"))
async def shop_response(message: types.Message, bot: Bot):
 """Магазин отвечает"""
 # Парсим: /reply_12345 620 в наличии
 parts = message.text.split()
 request_id = parts[0].split("_")[1]
 price = parts[1]
 availability = " ".join(parts[2:])

 # Сохраняем в БД/кэш
 await save_response(request_id, message.from_user.id, price, availability)

 await message.answer("Ваш ответ принят!")

async def aggregate_responses(request_id: str, timeout: int):
 """Собираем ответы и отправляем менеджеру"""
 await asyncio.sleep(timeout)

 responses = await get_responses(request_id)

 # Формируем отчет
 report = format_shop_responses(responses)

 await bot.send_message(
 chat_id=MANAGER_CHAT_ID,
 text=report,
 parse_mode="HTML"
)

```

## n8n Workflow для B2B

```

[Webhook: Telegram]
→ [Filter: команда /find]
→ [Supabase: получить список магазинов]
→ [Loop: по каждому магазину]
→ [Telegram: отправить запрос]
→ [Wait: 5 минут с накоплением ответов]
→ [AI: Gemini анализирует ответы]
→ [Telegram: отправить сводку менеджеру]

```

## Структура БД (Supabase)

```

-- Магазины
CREATE TABLE shops (
 id UUID PRIMARY KEY,
 telegram_chat_id BIGINT UNIQUE,
 name TEXT,
 distance_km INTEGER,
 rating DECIMAL(3,2),
 active BOOLEAN DEFAULT true
);

-- Запросы
CREATE TABLE part_requests (
 id UUID PRIMARY KEY,
 request_id TEXT UNIQUE, -- для broadcast
 query TEXT,
 manager_chat_id BIGINT,
 created_at TIMESTAMP DEFAULT NOW(),
 status TEXT -- pending, completed, timeout
);

-- Ответы магазинов
CREATE TABLE shop_responses (
 id UUID PRIMARY KEY,
 request_id TEXT REFERENCES part_requests(request_id),

```

```
shop_id UUID REFERENCES shops(id),
price DECIMAL(10,2),
availability TEXT,
response_time INTERVAL,
created_at TIMESTAMP DEFAULT NOW()
);
```

-- Индексы для быстрого поиска

```
CREATE INDEX idx_request_id ON shop_responses(request_id);
```

```
CREATE INDEX idx_active_shops ON shops(active) WHERE active = true;
```

## 📦 Обновленный технологический стек

### Финальная версия для MVP

AI & Agents:

- Gemini 2.0 Flash (основной)
- Gemini 1.5 Pro (резерв для сложных задач)
- Google AI Python SDK

Browser Automation:

- Playwright (ядро)
- MCP Browser Server (обертка)
- crawl4ai (для простых задач)

Backend:

- FastAPI (основное API)
- Python 3.11+
- Pydantic v2 (валидация)

Orchestration:

- n8n (визуальные workflow)
- MCP Protocol (интеграции)

Database:

- Supabase PostgreSQL
- Supabase Storage (файлы)

Messaging:

- aiogram 3.x (Telegram)

Infrastructure:

- Docker + Docker Compose
- Cloudflare Tunnel
- GitHub Actions (CI/CD)

Development:

- VS Code
- Continue / Cline
- MCP Inspector
- Gemini Code

## 📅 Скорректированный Roadmap

### Phase 1: MVP (фокус на основной функционал)

**Цель:** Автоматизация подбора запчастей для 1 автосервиса

#### Milestone 1.1: Инфраструктура (1 неделя)

- 📦 Создать GitHub репозиторий
- 🐳 Настроить Docker Compose
- 🔌 Подключить Supabase
- 🌐 Настроить Cloudflare Tunnel
- 🔄 Развернуть n8n

#### Milestone 1.2: Каталог (2 недели)

- □ Интеграция с exist.ru API/парсинг
- □ Получение артикулов по VIN/марке
- □ Базовая БД деталей

### **Milestone 1.3: Парсинг поставщиков (3 недели)**

- □ Playwright + MCP для 3-5 поставщиков
- □ Gemini агент для извлечения данных
- □ Кэширование в Supabase
- □ n8n workflow для координации

### **Milestone 1.4: AI Анализ (2 недели)**

- □ Gemini промпты для анализа
- □ Сравнение цен/наличия/сроков
- □ Генерация рекомендаций (Markdown → PDF)

### **Milestone 1.5: Telegram бот менеджера (1 неделя)**

- □ Базовые команды (/find, /status)
- □ Отправка отчетов
- □ Прямые ссылки на поставщиков

**Итого Phase 1: ~2 месяца**

## **Phase 2: B2B Сеть (упрощенная)**

### **Milestone 2.1: Telegram бот для магазинов (2 недели)**

- □ Регистрация магазинов
- □ Broadcast механизм
- □ Агрегация ответов (5 мин TTL)

### **Milestone 2.2: Интеграция с основным поиском (1 неделя)**

- □ Объединение результатов (онлайн + локальные магазины)
- □ Сортировка по скорости получения

**Итого Phase 2: ~3 недели**

## **□ Следующие шаги**

### **Немедленно (эта неделя)**

1. **Создать репозиторий**  
Название: partscout  
Структура: monorepo
2. **Определить 3-5 ключевых поставщиков**
  - Какие сайты мониторит менеджер сейчас?
  - API есть или только парсинг?

### 3. Настроить dev-окружение

- Docker Compose с n8n + Supabase
- Gemini API ключ

### 4. Создать первый MCP сервер

- Простой пример для отладки в VS Code

## На следующей сессии обсудим:

- ☐ Список поставщиков и их особенности
- ☐ Структура БД (детальная схема)
- ☐ Первый n8n workflow (схема)
- ☐ Промпты для Gemini (черновики)

## Готов к старту разработки? ☐

Какой пункт хочешь детализировать первым?